AWS Security Incident Response & Hardening Project

Sanjo Mathew Thomas

Introduction

As part of this project, I embarked on my first hands-on experience with **Amazon Web Services (AWS)**. My initial goal was to familiarize myself with AWS services and understand how they work together in a cloud environment. To achieve this, I started by setting up **AWS CloudTrail, VPC Flow Logs, GuardDuty, and Splunk** for security monitoring and threat detection. The primary objective of this project was to detect, analyze, and mitigate any security threats targeting an **AWS EC2 instance**, while also strengthening the overall security posture of the cloud environment.

During this project, I encountered **real-time security threats**, which gave me the opportunity to apply **AWS security best practices** and develop **incident response strategies**. The most significant incident involved repeated unauthorized access attempts from a **malicious IP address (124.220.134.127)**. This led me to investigate, mitigate, and set up continuous monitoring to prevent similar attacks in the future. Below is a detailed account of the steps taken throughout the project.

Setting Up AWS for Security Monitoring

Since this was my first experience with AWS, I began by **learning the fundamentals** and setting up key security services to monitor and analyze network activity. These included:

- **EC2 Instance Setup:** I launched an **EC2 instance** to simulate a real-world cloud environment and host applications.
- **CloudTrail Configuration:** AWS **CloudTrail** was enabled to log all API activity within the AWS environment, ensuring visibility into actions performed by users and services.
- **VPC Flow Logs Setup:** I activated **VPC Flow Logs** to capture and analyze network traffic to and from the EC2 instance. This helped in detecting unauthorized access attempts.
- **GuardDuty Activation:** AWS **GuardDuty** was configured to provide **threat intelligence and anomaly detection**, allowing me to detect suspicious activities automatically.
- **Splunk Integration:** I integrated **AWS logs with Splunk**, which allowed me to perform advanced **security monitoring, log analysis, and real-time alerting**.

With these services in place, I established a **security monitoring framework** capable of detecting and responding to potential threats effectively.

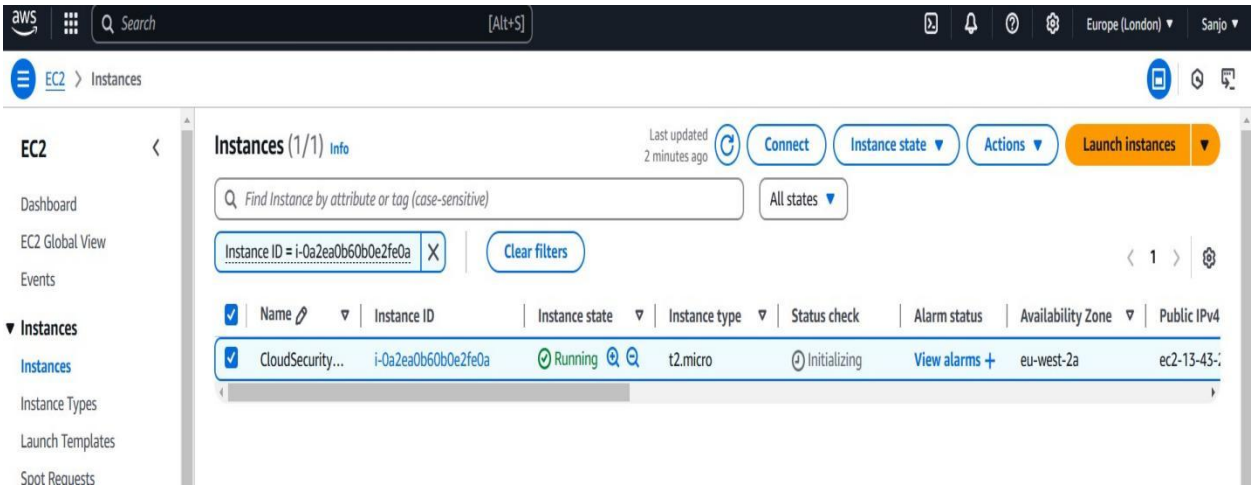

Image 1 : Ec2 Instance Setup

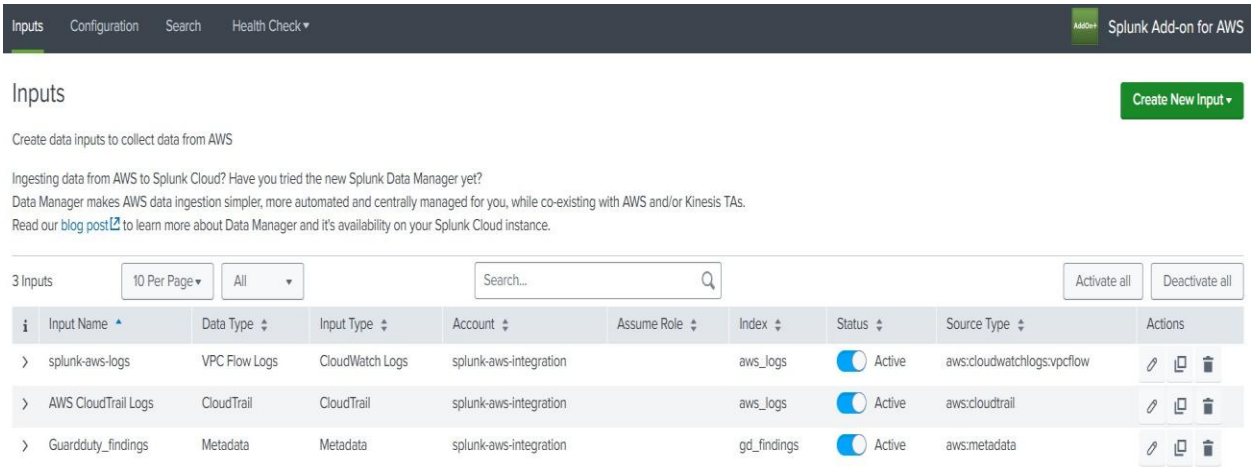

Image 2: Splunk Configuration Inputs

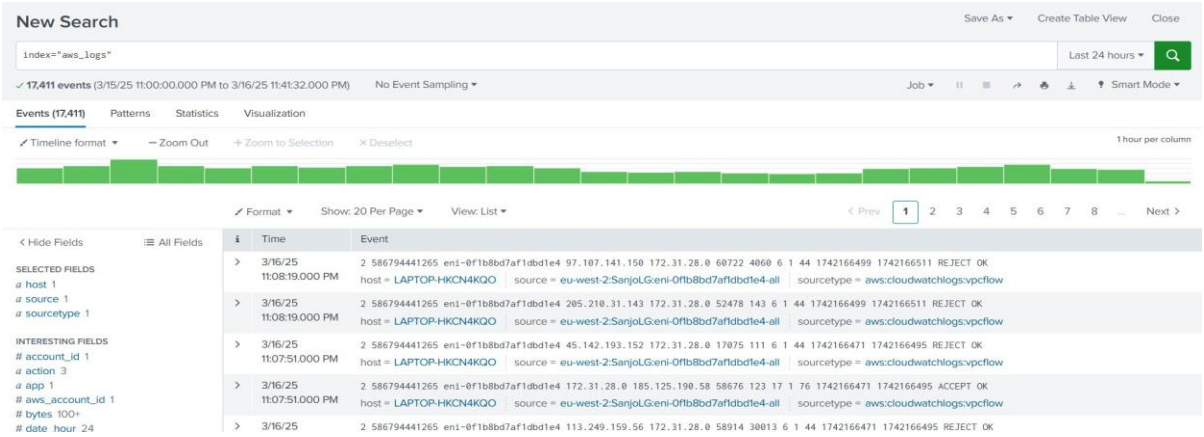

Image 3: Splunk Search result for VPC Flow Logs

Incident Detection & Analysis

After configuring AWS security tools, I began **analyzing incoming network traffic** using **VPC Flow Logs and Splunk**. It was during this process that I detected repeated connection attempts from **IP 124.220.134.127** to my **EC2 instance on port 80 (HTTP)**.

To confirm whether this was a **malicious activity or a false positive**, I used the following detection methods:

- **Reviewing AWS VPC Flow Logs**: I noticed multiple inbound requests from the same IP in a short time window.
- **Checking Splunk for Anomalies**: I ran queries to identify if the same IP had been making suspicious requests repeatedly.
- **Investigating GuardDuty Alerts**: GuardDuty flagged the IP as **potentially malicious**, confirming that this was not regular web traffic.
- **Verifying with Threat Intelligence Tools**: I checked the IP against **VirusTotal** and found that it had been reported for suspicious activities before.

At this point, it was evident that the **IP was performing reconnaissance or a port-scanning attack**, attempting to gather information about my EC2 instance.

Image 4: Detection of Malicious IP

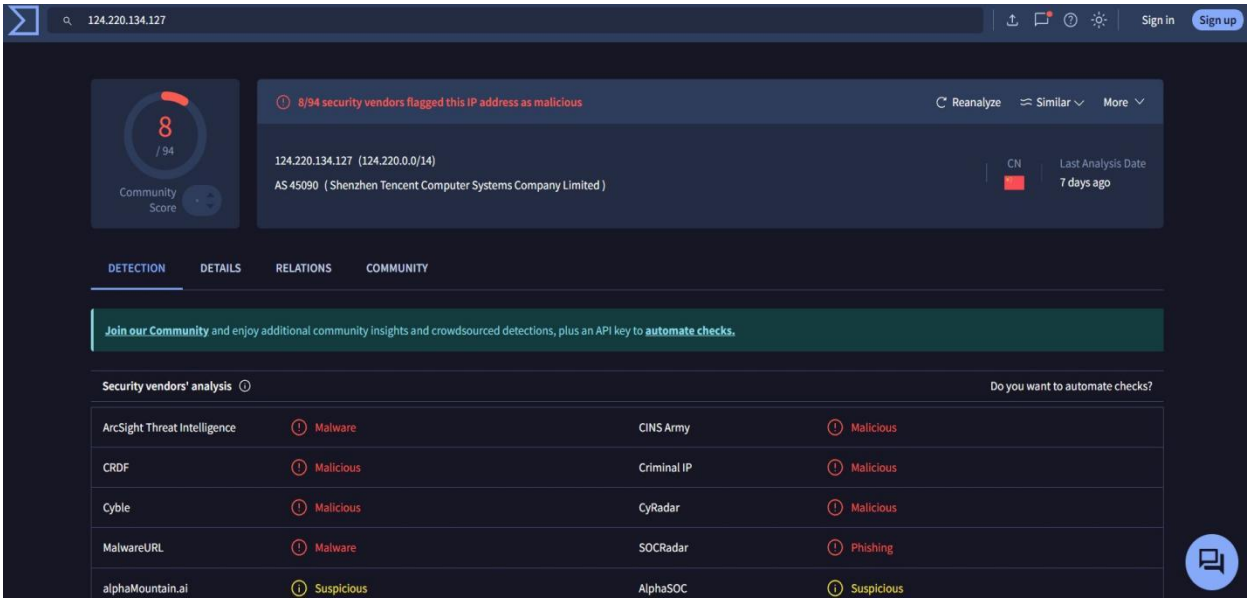

Image 5: Virus Total Cross Check

Mitigation: Blocking the Malicious IP

After confirming the threat, my next step was to **immediately block the malicious IP to prevent further unauthorized access attempts**. I implemented **multiple layers of security controls** to ensure that the attacker could not bypass any single defense mechanism.

1. **Blocking in AWS Security Groups**:
    a. I modified the **EC2 Security Group rules** to **deny all incoming traffic** from **124.220.134.127**.

2. **Blocking in AWS Network ACLs**:
    a. To provide **an extra layer of protection**, I created a **Deny rule** in the **Network ACL** associated with the EC2 instance's subnet, preventing both **inbound and outbound communication** with the malicious IP.
3. **Blocking in AWS WAF (Web Application Firewall)**:
    a. I configured **AWS WAF** by creating an **IP Set** that included **124.220.134.127** and linked it to a **Web ACL rule** to block any future attempts at the application layer.

After implementing these security controls, I verified that the **IP was no longer able to communicate** with my AWS environment by checking **AWS logs and running a new Splunk query**. The logs confirmed that the attack had been successfully mitigated.

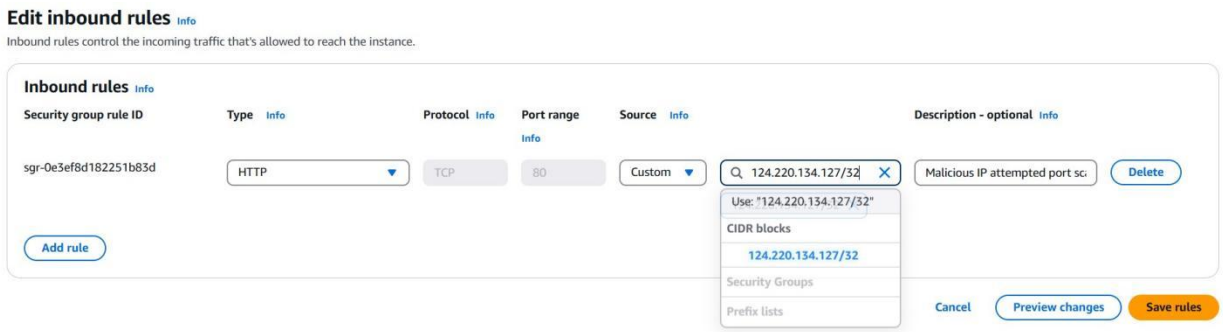

Image 6: Setting Inbound Rules

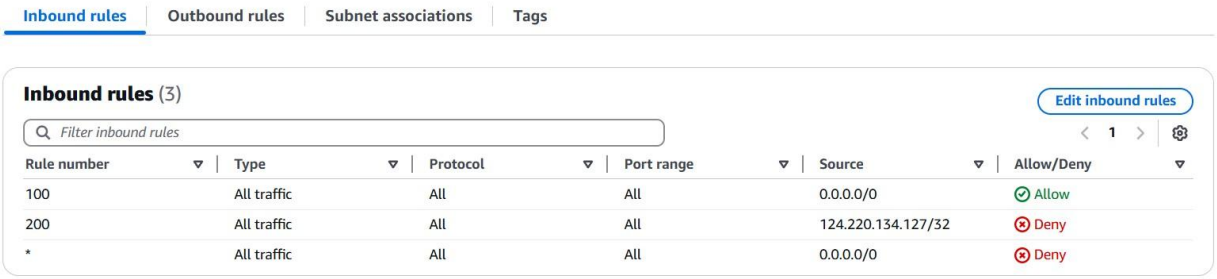

Image 7: Network ACL rules

Validation: Running an Nmap Scan

To ensure that my security controls were effectively blocking unauthorized access, I decided to **perform an Nmap scan** on my EC2 instance.

- I ran the following **Nmap command** to check which ports were open:

nmap -p 80 -Pn <ec2_ip>

- **Findings:**
  - The EC2 instance **was up**, confirming that it was still operational.
  - **Port 80 was marked as "filtered"**, indicating that the firewall rules were successfully blocking traffic.
  - No other open ports were detected, meaning that the attack surface was minimized.

By **validating the security measures**, I confirmed that the **mitigation steps were effective** in preventing further threats.

Image 8: Nmap Scan

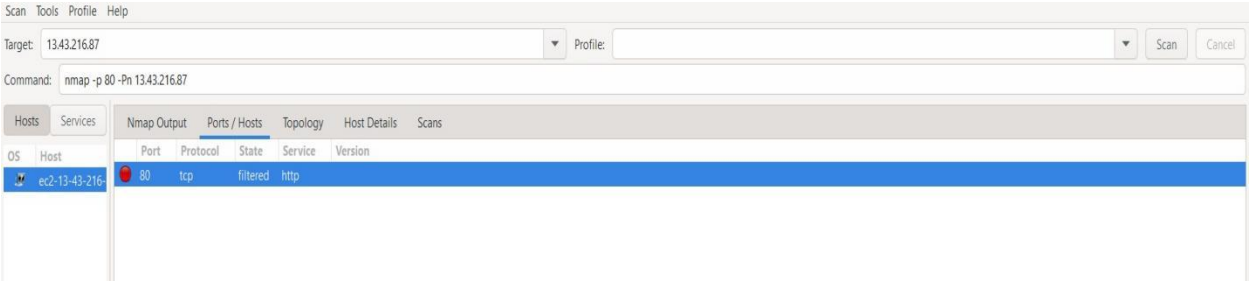

Image 9:Nmap Scan port 80 checking

Continuous Monitoring & Setting Up Splunk Alerts

To **enhance security monitoring and enable real-time threat detection**, I created **Splunk alerts** that would automatically notify me if similar attacks occurred in the future.

Splunk Query for Port Scanning Detection

I wrote a **Splunk query** to detect multiple connection attempts from the same IP:

index=aws_logs vpcflow_action=ACCEPT dest_port=80 | stats count by src_ip | where count > 10

- This **query filters accepted traffic**, groups it by **source IP**, and flags **any IP that accesses port 80 more than 10 times in 5 minutes**.

Configuring Splunk Alerts

- **Alert Name:** Possible Port Scanning Detected
- **Trigger Condition:** If an IP makes **more than 10 requests in 5 minutes**.
- **Action:** Send an **email notification** and log the event in **Splunk Enterprise Security (ES)**.

By setting up this alert, I ensured that **any future suspicious activity would be detected in real-time**, allowing for a **quick response to security threats**.

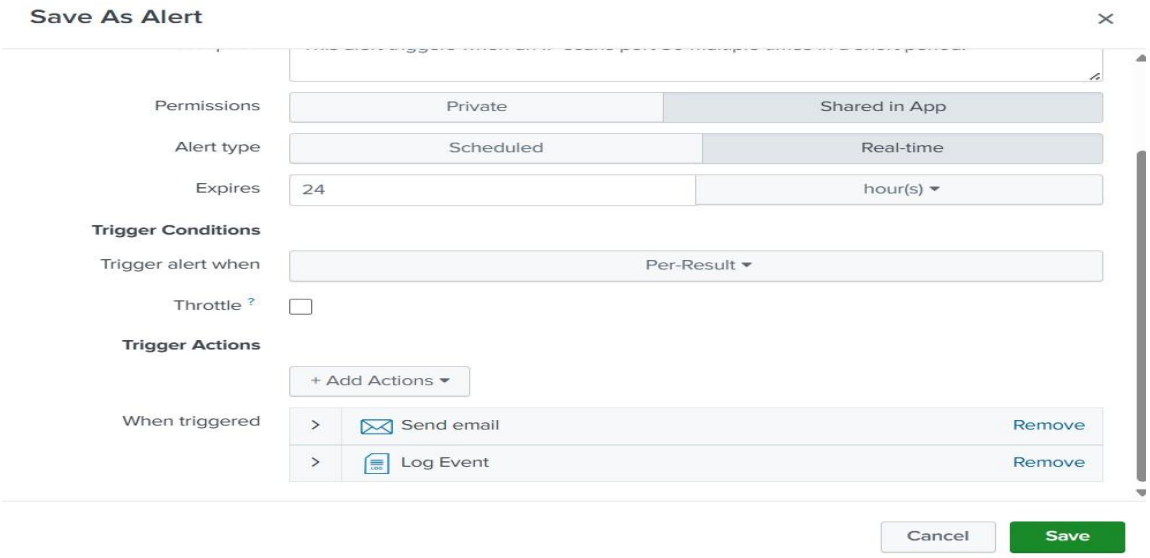

Image 10: Setting up alert in splunk

Incident Playbook & Final Documentation

To ensure that similar incidents could be handled efficiently in the future, I created an **Incident Response Playbook**. This document outlined the **step-by-step process** for:

- **Detecting threats using AWS logs & Splunk**
- **Investigating suspicious activities (Nmap scans, log analysis)**
- **Mitigating attacks by blocking IPs in Security Groups, ACLs, and WAF**
- **Validating security controls to confirm threats are neutralized**
- **Implementing continuous monitoring & alerting for real-time detection**

Additionally, I prepared a **comprehensive final report** with **screenshots of AWS configurations, Splunk alerts, and Nmap scan results** to serve as **documentation for security audits and compliance**.

Lessons Learned & Security Best Practices

This project provided **valuable hands-on experience** in **AWS security monitoring, threat detection, and incident response**. The key takeaways from this experience include:

- **AWS security tools (Flow Logs, CloudTrail, GuardDuty) are essential for cloud monitoring.**
- **Blocking threats at multiple layers (Security Groups, ACLs, WAF) enhances security.**
- **Real-time Splunk alerts significantly improve incident response times.**
- **Regular security assessments, such as Nmap scans, help validate firewall rules.**

Moving forward, I plan to **continue improving my AWS security skills**, explore **automated threat response mechanisms**, and conduct **regular security audits** to maintain a robust cloud security posture.

Conclusion

This project successfully **enhanced AWS security** by setting up **threat detection, implementing security controls, and responding to real-world incidents**. Through **learning, experimentation, and problem-solving**, I gained **valuable cloud security expertise** that will help me in future projects.

Final Status: Incident Resolved – AWS Security Improved